REPORTE TAREA 1
ALGORITMOS Y COMPLEJIDAD

«Más allá de la notación asintótica: Análisis experimental de algoritmos de ordenamiento y multiplicación de matrices.»

Benjamin Campos

10 de abril de 2026

00:50

1. Entorno experimental

Hardware y Software:

- **Procesador:** Intel Core i5 10ª generación
- **Memoria RAM:** 8 GB
- **Sistema Operativo:** Arch Linux
- **Compilador:** GCC 15.2.1
- **Entorno:** Laptop Asus VivoBook
- **IDE:** Caelestia

Para realizar las pruebas de ordenamiento de Arrays se utilizaron arreglos unidimensionales con tamaños $n \in \{10^1, 10^3, 10^5, 10^7\}$ en configuraciones aleatorias, ascendentes y descendentes y para la multiplicación de matrices, se emplearon matrices cuadradas (n x n) de dimensiones $n \in \{2^4, 2^8, 2^9, 2^{10}\}$ con tipos densa, dispersa y diagonal.

2. Resultados y Análisis

2.1. Resultados de la Multiplicación de matrices

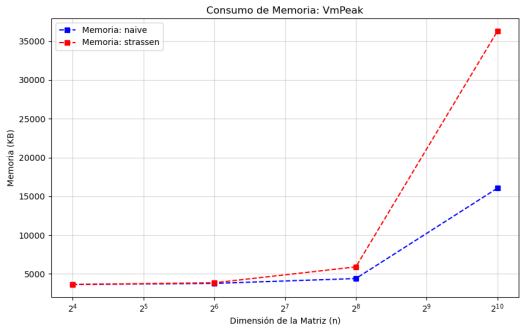

Figura 1: Comparativa de memoria.

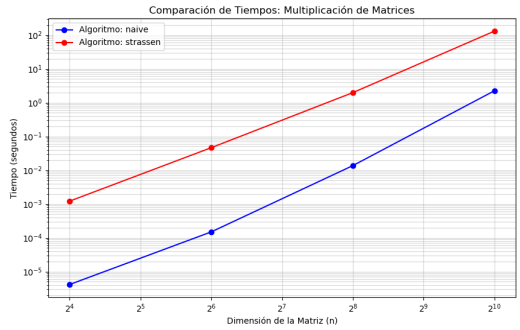

Figura 2: Comparativa de tiempo.

Dimensión (n)	Naive (s)	Strassen (s)
2^4	4×10^{-6}	1×10^{-3}
2^6	$1{,}5 \times 10^{-4}$	5×10^{-2}
2^8	$1{,}2 \times 10^{-2}$	2
2^{10}	2	120

Cuadro 1: Tiempo de ejecución en multiplicación de matrices

Dimensión (n)	Naive (KB)	Strassen (KB)
2^4	3500	3500
2^6	3800	3800
2^8	4500	6000
2^{10}	16000	36000

Cuadro 2: Consumo de memoria en multiplicación de matrices

Al observar el comportamiento de los dos algoritmos implementados en términos de ejecución, se ve a simple vista que el algoritmo Naive es mucho más eficiente técnicamente que el algoritmo Strassen para las dimensiones probadas. Se sabe que la teoría asintótica sugiere que Strassen debería ser más eficiente debido a su complejidad, la cual es de $O(n^{2,81})$ en comparación a la de Naive, la cual es de $O(n^3)$. Si nos fijamos en el Cuadro 1 (Tiempo de ejecución en multiplicación de matrices), esto nos muestra una realidad completamente distinta a lo esperado, ya que para una matriz de 2^{10}, Naive resuelve la operación en tan solo 2 segundos mientras que Strassen se demora 120 segundos. Esta diferencia se puede explicar por la constante oculta que contiene Strassen, ya que la reducción de una multiplicación entre matrices contiene 18 operaciones de suma y resta adicionales en cada nivel de recursión que se ejecuta. Estas ope-

raciones resultan contraproducentes en matrices de tamaño mediano, debido a que el costo de gestionar estas operaciones aritméticas extras y el overhead de las múltiples llamadas recursivas superan con creces el beneficio de reducir la complejidad polinomial. Ahora, si nos fijamos en la Figura 1 (Comparativa de memoria), nos muestra unos resultados preocupantes si se desea usar en este caso el algoritmo Strassen en entornos de computación con recursos limitados o usar mayor tamanos de matrices. Mientras que por el lado de Naive el algoritmo muestra una tendencia lineal y predecible de memoria, Strassen muestra un crecimiento abrupto que ha llegado a duplicar su consumo al alcanzar una dimensión de $n = 2^{10}$ (aprox. 35.000 KB en comparación a 16.000 KB). Esto es debido a la naturaleza del método (divide y vencerás), ya que mientras la multiplicación tradicional opera de una manera eficiente en términos de espacio, Strassen usa la creación y almacenamiento de siete matrices intermedias (M_1 a M_7) en cada etapa del proceso. Esto no solo eleva el uso de memoria, sino que también degrada el rendimiento debido a la pérdida de la localidad de la memoria, obligando al sistema a realizar constantemente operaciones de asignación en el heap. Aplicando el Teorema Maestro al algoritmo de Strassen $T(n) = 7T(n/2) + \Theta(n^2)$, obtenemos una complejidad de $O(n^{\log_2 7}) \approx O(n^{2,81})$. No obstante, el análisis experimental revela que el costo de las 18 operaciones adicionales de suma/resta por nivel y la gestión de las 7 matrices intermedias imponen un factor constante tan elevado que la ventaja teórica no se percibe en las dimensiones probadas.

2.2. Algoritmos de ordenamiento

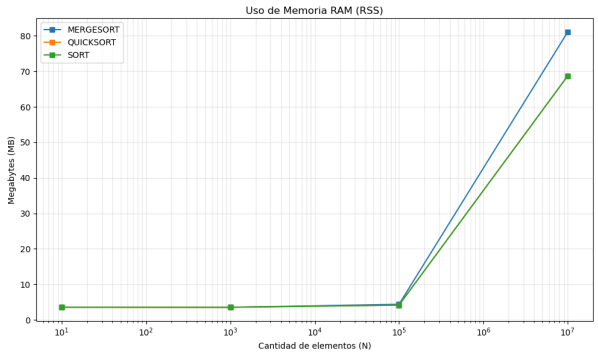

Figura 3: Comparativa de memoria.

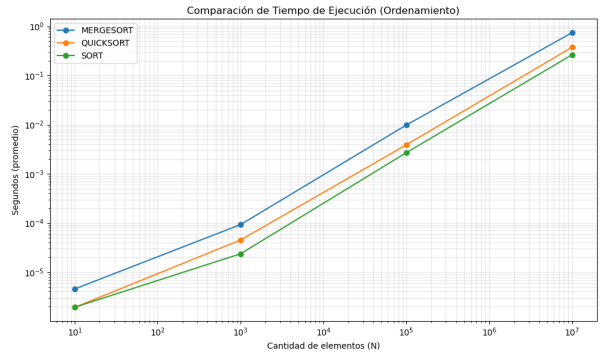

Figura 4: Comparativa de tiempo.

N	MergeSort (s)	QuickSort (s)	Sort (s)
10	5×10^{-6}	2×10^{-6}	2×10^{-6}
10^3	1×10^{-4}	5×10^{-5}	2×10^{-5}
10^5	1×10^{-2}	4×10^{-3}	3×10^{-3}
10^7	0,8	0,3	0,25

Cuadro 3: Tiempo de ejecución de algoritmos de ordenamiento

N	MergeSort (MB)	QuickSort (MB)	Sort (MB)
10	3.5	3.5	3.5
10^3	3.6	3.6	3.6
10^5	4.5	4.5	4.5
10^7	81	69	69

Cuadro 4: Uso de memoria en algoritmos de ordenamiento

Para el caso de los arreglos, se utilizaron tres algoritmos distintos en comparación con la multiplicación de matrices: se trabajó con sort, Quicksort y Mergesort. Si observamos el Cuadro 3 y la Figura 4, se puede apreciar que, aunque teóricamente los tres algoritmos comparten una complejidad promedio de $O(n \log n)$, existe una diferencia significativa en su rendimiento práctico. La librería sort, al ser el estándar de C++, demuestra ser la más eficiente en los rangos probados, alcanzando un tiempo de 0,25 segundos para $n = 10^7$. Le sigue Quicksort con un tiempo de 0,3 segundos; esta diferencia se debe a que la librería estándar suele emplear algoritmos híbridos (como Introsort), lo que optimiza el proceso al cambiar de estrategia según el tamaño de la sublista y aprovechar mejor las instrucciones a nivel de hardware. Por otro lado, Mergesort resulta ser el más lento en todos los casos probados, llegando a los 0,8 segundos para $n = 10^7$. Respecto al uso de memoria, el Cuadro 4 y la Figura 3 revelan un comportamiento elevado en términos de escalabilidad para grandes volúmenes de datos; se observa que hasta $n = 10^5$, el consumo de los tres algoritmos se mantiene estable y similar entre sí (aprox. 4,5 MB). Sin embargo, al alcanzar $n = 10^7$ se produce un salto drástico: mientras Mergesort eleva su consumo hasta los 81 MB, Quicksort y sort se mantienen en 69 MB. Esta disparidad se debe a la principal desventaja de Mergesort: su naturaleza no es in-place, por lo que requiere arreglos auxiliares proporcionales al tamaño de la entrada para realizar la mezcla de los elementos. En cambio, Quicksort y sort optimizan el uso de espacio al trabajar, en su gran mayoría, sobre el mismo arreglo de entrada. Para validar estos resultados desde una perspectiva formal, se puede aplicar el Teorema Maestro a la recurrencia de Mergesort, definida como $T(n) = 2T(n/2) + \Theta(n)$. En este caso, al tener $a = 2$, $b = 2$ y $f(n) = n$, se cumple que $f(n) = \Theta(n^{\log_2 2})$, lo que corresponde al Caso 2 del teorema, confirmando una complejidad de $\Theta(n \log n)$. Aunque Quicksort y sort (Introsort) comparten esta cota superior teórica, la experimentación demuestra que las constantes de implementación y la gestión de memoria in-place son los factores determinantes en la eficiencia real sobre el hardware.

3. Conclusiones

Considerando los resultados obtenidos, se concluye que, para las dimensiones evaluadas, la eficiencia práctica está fuertemente condicionada por la gestión de memoria y el overhead algorítmico más que por la complejidad teórica. En el ordenamiento de arreglos, la librería estándar std::sort $(0,25$ s) superó ampliamente a Mergesort $(0,8$ s) debido a su naturaleza in-place y al uso de algoritmos híbridos que optimizan el hardware, evitando el uso excesivo de memoria (69 MB frente a 81 MB para $n = 10^7$). Por otro lado, en la multiplicación de matrices, el algoritmo Naive resultó ser drásticamente más eficiente (2 s) y

ligero (16.000 KB) que Strassen (120 s y 35.000 KB) para $n = 2^{10}$. Esto evidencia que, en matrices de tamaño mediano, las constantes ocultas y las múltiples operaciones de suma, reducción y asignación en memoria del método de Strassen contrarrestan cualquier ventaja competitiva de su complejidad asintótica $O(n^{2,81})$.